



Университет ИТМО
Мегафакультет компьютерных технологий и управления
Факультет программной инженерии и компьютерной техники

Отчет по лабораторной работе №4

по дисциплине «бизнес-логика программных систем»

Вариант: 3212

Выполнили:
Григорий Садовой
Байрамгулов Мунир

Преподаватель:
Кривоносов Егор Дмитриевич

1 Текст задания

Внимание: у разных вариантов разный текст задания.

Переработать программу, созданную в результате выполнения лабораторной работы №3, следующим образом:

- Для управления бизнес-процессом использовать BPM-движок Camunda.
- Заменить всю «статическую» бизнес-логику на «динамическую» на базе BPMS. Весь бизнес-процесс, реализованный в ходе выполнения предыдущих лабораторных работ, включая разграничение доступа по ролям, управление транзакциями, асинхронную обработку и периодические задачи, должен быть сохранен.
- BPM-движок должен быть запущен в режиме standalone-сервиса.
- Для описания бизнес-процесса необходимо использовать Camunda Modeler.
- Пользовательский интерфейс приложения должен быть сгенерирован с помощью генератора форм Camunda.
- Итоговая сборка должна быть развернута на сервере Helios под управлением сервера приложений WildFly.

Правила выполнения работы:

- Описание бизнес-процесса необходимо реализовать на языке BPMN 2.0.
- Необходимо интегрировать в состав процесса, управляемого BPMS, все, что в принципе возможно в него интегрировать. Если какой-то из компонентов архитектуры приложения не поддерживается напрямую, необходимо использовать для интеграции с этой подсистемой соответствующие API и адаптеры.
- Распределенную обработку задач и распределенные транзакции на BPM-движок переносить не требуется.

2 Краткое описание предметной области

Автоматизируемый бизнес-процесс относится к системе подбора персонала. В процессе участвуют три основные роли:

- CANDIDATE – кандидат, который просматривает вакансии, отправляет отклик и отвечает на приглашение.
- RECRUITER – рекрутер, который создает вакансии, рассматривает отклики, приглашает кандидатов на интервью, отменяет интервью и закрывает вакансии.
- ADMIN – администратор, который выполняет административные операции и проверяет просроченные приглашения.

Ключевой сценарий: рекрутер создает вакансию, кандидат отправляет отклик, система выполняет автоматический скрининг, рекрутер принимает решение, кандидат получает уведомление и отвечает на приглашение. Дополнительно реализованы закрытие вакансии, обработка просроченных приглашений, уведомления и разграничение доступа в Camunda Tasklist.

3 Модель потока управления

Основная модель потока управления перенесена в BPMN 2.0 и выполняется в Camunda. Статическая серверная логика лабораторной работы №3 заменена управлением через BPMN-процессы, DMN-таблицы решений, Camunda Forms и external task worker приложения.

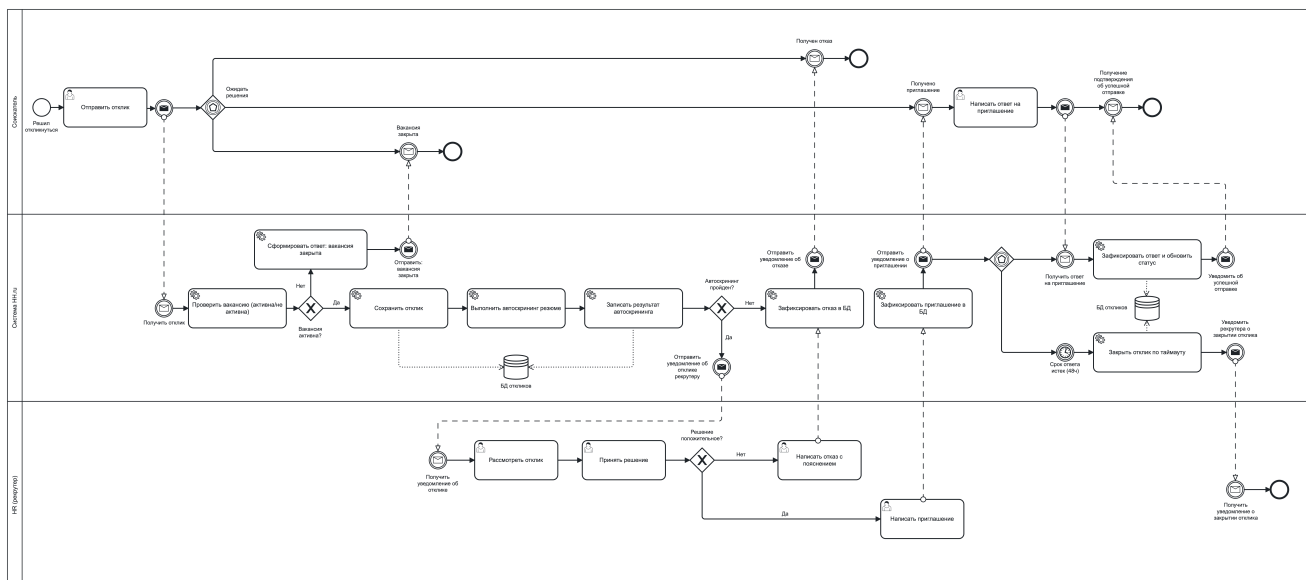


Рис. 1. Модель потока управления бизнес-процессом обработки откликов и вакансий

3.1 Основные BPMN-процессы

Process key	Назначение	Роли запуска
hhVacancyProcess	Создание и закрытие вакансии, изменение связанных откликов и уведомление кандидатов.	RECRUITER
hhApplicationProcess	Подача отклика, автоскрининг, решение рекрутера, приглашение, ответ кандидата и завершение отклика.	CANDIDATE
hhRecruiterInterviewCancelProcess	Отмена интервью рекрутером с возвратом отклика на рассмотрение и уведомлением кандидата.	RECRUITER
hhVacancyStatusUpdateProcess	Изменение статуса вакансии с проверкой разрешенного перехода через DMN.	RECRUITER
hhTimeoutSchedulerProcess	Периодическая обработка просроченных приглашений.	ADMIN
hhAdminInterviewResetProcess	Административный сброс интервью и возврат отклика на рассмотрение.	ADMIN
hhNotificationProcess	Переиспользуемый подпроцесс формирования уведомлений.	Вызывается из других процессов

3.2 UI-процессы Camunda Tasklist

Для демонстрации пользовательских экранов через Camunda Forms добавлены отдельные UI-процессы:

- hhUiCandidateVacancyList – экран вакансий кандидата.
- hhUiCandidateApplicationList и hhUiCandidateApplicationView – список и карточка отклика кандидата.
- hhUiRecruiterVacancyList – вакансии рекрутера.
- hhUiRecruiterApplicationList и hhUiRecruiterApplicationView – список и карточка отклика для рекрутера.
- hhUiRecruiterSchedule – расписание интервью рекрутера.
- hhUiNotificationList – уведомления текущего пользователя.
- hhUiAdminTimeoutReview – административный запуск проверки таймаутов.

Эти процессы нужны не для замены REST API, а для демонстрации того, что пользовательские действия доступны в Camunda Tasklist и ограничены ролями.

3.3 DMN-таблицы

DMN	Назначение
hhAutoScreening	Решает, прошел ли отклик автоматический скрининг по разнице между баллом резюме и порогом вакансии.
hhStatusTransitions	Описывает допустимые переходы статусов вакансии и отклика.
hhOperationPermissions	Проверяет разрешения операций по роли и владению объектом.
hhNotificationTemplates	Выбирает тип и шаблон уведомления для кандидата или рекрутера.

4 Блок-схема архитектуры приложения

Пользователь Tasklist / REST	->	Spring Boot WAR WildFly Helios	->	PostgreSQL данные приложения
		Camunda adapter External Task Worker		Camunda Engine standalone service
		Narayana JTA транзакции приложения	->	БД и расписание интервью

Таблица 1. Блок-схема архитектуры и основные интеграционные связи

4.1 Точки интеграции Camunda с подсистемами

Точка интеграции	Как используется	Подсистема приложения
REST API Camunda Engine	Деплой BPMN/DMN/Form, запуск процессов, корреляция сообщений, работа с задачами и авторизациями.	Camunda standalone service
External Tasks	Spring Boot worker забирает задачи по topic и выполняет бизнес-операции вне движка.	Сервисы вакансий, откликов, интервью, уведомлений
Camunda Forms	Формы создания вакансии, отклика, решения рекрутера, приглашения, ответа кандидата и UI-экранов.	Camunda Tasklist
DMN	Автоскрининг, проверка прав, переходы статусов и выбор шаблонов уведомлений.	Camunda DMN engine
Message correlation	События закрытия вакансии, отмены интервью, административного сброса и истечения приглашения возвращают процесс в нужную ветку.	Camunda Runtime API

Tasklist filters and authorizations	Кандидат видит только кандидатские процессы, рекрутер – рекрутерские, администратор – административные.	Camunda Identity / Authorization API
Narayana JTA	Составные операции остаются транзакционными на стороне приложения, а BPMN явно показывает этапы и компенсации.	WildFly, PostgreSQL

5 Изменения относительно лабораторной работы №3

Пункт	Описание
Перенос управления процессом в Camunda	В лабораторной работе №3 сценарии были в основном статическими сервисными цепочками приложения. В текущей версии порядок выполнения вынесен в BPMN 2.0: создание вакансии, отклик, автоскрининг, решение рекрутера, приглашение, ответ кандидата, закрытие вакансии и таймауты управляются BPMS.
Вынос решений в DMN	Проверка автоскрининга, допустимости переходов статусов, прав операции и шаблонов уведомлений оформлена таблицами решений. Это делает правила изменяемыми без переписывания Java-кода.
Camunda Forms вместо ручных экранов	Для действий в Tasklist добавлены формы Camunda: создание вакансии, отклик, решение рекрутера, приглашение, ответ кандидата, закрытие вакансии, просмотр списков и уведомлений.
Внешний адаптер вместо Java task listeners в движке	Бизнес-валидация и работа с БД выполняются Spring Boot worker через external tasks. Это сохраняет границу ответственности: Camunda управляет процессом, Java-приложение выполняет бизнес-операции.
Ролевые ограничения в Camunda	Пользователи и группы синхронизируются в Camunda. Start process и Tasklist-фильтры ограничены группами CANDIDATE, RECRUITER, ADMIN.
BPMN error loop для форм	При ошибке server-side валидации external task бросает BPMN error, процесс возвращается на форму, а пользователю показываются formErrorMessage и список ошибочных полей.
Компенсации и транзакционные блоки	Составные операции закрытия вакансии, приглашения, ответа кандидата, отказа и таймаута описаны как транзакционные участки с компенсационными ветками. Распределенные транзакции остаются в Narayana, как требует задание.
Standalone Camunda на Helios	Camunda запущена отдельным сервисом, приложение развернуто как WAR в WildFly, взаимодействие идет через REST API Camunda Engine.
Очистка окружения при деплое	Helios bootstrap удаляет старые runtime/history-инстансы управляемых hh* процессов перед новым деплоем, чтобы демонстрация не смешивалась со старыми запусками.

6 Исходный код системы

Исходный код системы, BPMN/DMN/Form-описания и скрипты деплоя находятся в репозитории:

<https://github.com/msuny-c/hh-process>

Основные директории:

- `src/main/resources/camunda` – BPMN 2.0, DMN и формы Camunda.
- `src/main/java/ru/itmo/hhprocess/camunda` – интеграция с Camunda REST API, deployment service, external task worker, синхронизация identity и authorizations.
- `src/main/java/ru/itmo/hhprocess/service` – бизнес-сервисы приложения.
- `scripts/helios-bootstrap-camunda.sh` – bootstrap пользователей, групп, фильтров, авторизаций и очистка старых процессов на Helios.
- `.github/workflows/deploy-split-helios.yml` – CI/CD деплой Camunda standalone и WAR-приложения в WildFly.

7 Проверка работоспособности

Для проверки используются smoke-тесты API, Camunda-level smoke flow и интеграционные тесты модели:

```
python3 test/test_camunda_model_coverage.py
python3 test/test_camunda_visual_model_contract.py
mvn test
```

В демонстрационном сценарии проверяются:

- создание вакансии рекрутером;
- автоскрининг и автоотказ кандидату;
- успешный отклик и приглашение на интервью;
- ответ кандидата на приглашение;
- закрытие вакансии и уведомление кандидата;
- работа планировщика просроченных приглашений;
- отображение завершенных и активных экземпляров процесса в Camunda Cockpit.

8 Выводы по работе

В ходе лабораторной работы приложение было переработано для управления бизнес-процессом через Camunda. Управляющая логика вынесена в BPMN 2.0 и DMN, пользовательские действия доступны через Camunda Tasklist и Camunda Forms, а бизнес-операции выполняются внешним Spring Boot adapter worker через external tasks.

Архитектура сохранила требования предыдущих лабораторных работ: разграничение доступа по ролям, транзакционность составных операций, асинхронную обработку и периодические задачи. При этом процесс стал наглядным: в Cockpit можно увидеть текущий узел выполнения, историю, переменные, ошибки и завершение экземпляров процесса.

Разделение ответственности улучшило сопровождаемость системы. Camunda отвечает за маршрут процесса, роли, пользовательские задачи, таймеры и решения, а приложение отвечает за проверку бизнес-правил, работу с БД, JTA-транзакции и интеграцию с доменной моделью.